

OSTAD · BATCH 08 · LIVE CLASS 9:00 – 10:30

Authentication, Login & Registration

How this LMS works out who you are, proves it on every request, and decides what you are allowed to do.

Django 5.2.7

DRF 3.15.2

Simple JWT 5.4.0

React 19

React Router 7

PBKDF2

Bearer tokens

Five parts, and where the clock should be

PART	CLOCK	WHAT WE COVER	BOOK CHAPTERS
1	0 – 18 min	Foundations. Authn vs authz, why HTTP forgets you, where a person is stored	Ch 1 – 2
2	18 – 38 min	Making an account. Registration, serializers, password hashing	Ch 3 – 4
3	38 – 60 min	Login and the token. The view line by line, then what a JWT actually is	Ch 5 – 6
4	60 – 80 min	Staying in. The browser half, then permissions and 401 vs 403	Ch 7 – 8
5	80 – 90 min	Passwords again, gaps, recap. Change vs reset, what this project does not do	Ch 9 – 12

GROUND RULE

Stop me the moment a word is unfamiliar. Every term in this deck is defined the first time it appears — if it wasn't, that's my bug.

Two questions, in this order

THE QUESTION	ITS NAME	IN THIS APP
Who are you, and can you prove it?	Authentication authn	You type a phone number and a password. The server checks them and hands you a token.
Now that I know who you are, what may you do?	Authorisation authz	Your account has a role — admin, teacher or student — and each role may change different things.

Authentication happens properly **once**, at login. Authorisation happens on **every single request afterwards**. A signed-in student and a signed-in admin are equally authenticated; they are not equally authorised.

Chapters 2–7 are authn. Chapter 8 is authz.

PART ONE

Foundations

- The doorman, the badge, and the card reader
- Why HTTP forgets you between requests
- Sessions vs tokens, and what this project chose
- The nine-line story, and the nine files
- An account is **two rows**, not one



The doorman and the floor plan

At the front desk

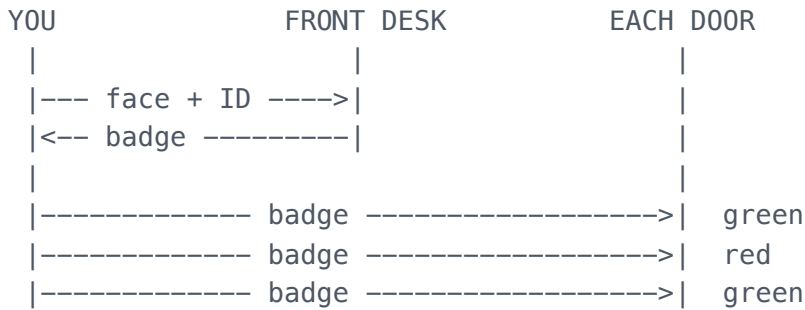
A doorman checks your face against your ID card. He is satisfied you are who you say you are, and gives you a plastic badge on a lanyard.

That is authentication. It happens once.

At each door

The badge does not open every door. The server-room reader beeps red at a visitor badge and green at an engineer's. Nobody re-examines your face — the badge speaks for you.

That is authorisation. It happens at every door.



NOT A SIMPLIFICATION

The **token** login hands you is that badge. The **permission classes** on each API endpoint are the card readers on each door. This is genuinely how the code is arranged.

The web forgets you instantly

HTTP is stateless

The server keeps **no memory** between one request and the next. Each request arrives as a complete stranger.

Log in, then a second later ask for the course list — as far as the server is concerned, two unrelated strangers knocked on the door.

The upside: a stateless server can be restarted or duplicated across ten machines and nothing personal is lost, because nothing personal was being held.

So every request that needs to know who you are has to **carry the proof with it**. Not "log in and be remembered" — log in, get proof, and **re-present that proof every single time**.

Twenty API calls means twenty presentations of the badge.

COMMON TRAP

"I logged in, so why does the server say 401?" Almost always: the request went out **without the proof attached**. Being logged in is not a state the server holds. It is a piece of data your browser has to send.

A claim ticket, or a signed note

	SESSION + COOKIE	TOKEN (JWT)
The proof is	A claim ticket . Means nothing alone; the meaning is in the server's table.	A signed statement . It already says who you are, and cannot be altered.
Server stores anything?	Yes — one row per session	No
Sent by the browser	Automatically	Only if your code attaches it
Logging one person out	Delete the row. Instant.	Hard. We come back to this.
Works well for	A server rendering its own HTML pages	A separate frontend, or a phone app, talking to an API
Used here for	Django's <code>/admin/</code> site	Everything the React app does

This is why you can be signed into `/admin/` in one tab and signed out of the React app in another. Different mechanisms.

Why tokens are the natural fit here

```
React frontend      Django backend
localhost:5180      127.0.0.1:8001
```

```
|      |
|  fetch() + Bearer token  |
|----->|
|<----- JSON -----|
```

Two separate programs. The frontend is NOT pages produced by Django – it is its own app that phones the backend for data.

Three consequences you will meet today

- The frontend must **attach the token by hand** to every call. Nothing is automatic.
- Two addresses means the browser's same-origin rule applies — which is why `corsheaders` is installed. CORS is the server saying "requests from that other address are fine by me".
- There is **no real "log out everywhere"** button, because there is no session table to empty.

Everything today, in nine lines

1. An admin fills in the Accounts form. -> POST /api/register/
2. Django hashes the password, saves a User row and a Profile row
 holding the phone number and the role.
3. The new person opens /login and types
 their phone number and password. -> POST /api/login/
4. Django finds the Profile by phone, checks the password against
 the stored hash.
5. Django signs two tokens and sends them back, along with the role.
6. The browser stores the access token in localStorage.
7. Every later call attaches it as a header:
 Authorization: Bearer <token>
8. Django verifies the signature, sets request.user, and the
 endpoint's permission class checks the role.
9. Eight hours later the token expires and step 3 happens again.

Every remaining slide today is one of these nine lines, slowed down.

Nine files do all of it

FILE	ITS JOB IN ONE LINE	PART
backend/backend/models.py	The <code>Profile</code> table: phone number and role	1
backend/backend/serializers.py	Checks and shapes what comes in and goes out	2, 5
backend/backend/views.py	The register, login, profile and password endpoints	2, 3, 5
backend/backend/urls.py	Which web address reaches which view	2
backend/backend/permissions.py	Which role may change what	4
backend/lms/settings.py	Token lifetimes, password rules, the default lock	3, 4
frontend/src/api.js	Stores the token, attaches it to every call	4
frontend/src/AuthContext.jsx	Holds "am I logged in" for the whole React app	4
frontend/src/components/ProtectedRoute.jsx	Bounces signed-out visitors to <code>/login</code>	4

An account is two rows, in two tables

auth_user			backend_profile	
-----			-----	
id	7		id	4
username	nadia	<-----	user_id	7
password	pbkdf2_sha256\$		phone	01711110001
email	nadia@...		role	teacher
is_active	True			

Django ships the left one

You did not write `User`. It arrives with `django.contrib.auth`. What it does **not** carry: a phone number, and a role. Django has no opinion about either.

backend/backend/serializers.py

```
from django.contrib.auth import get_user_model

User = get_user_model()
```

`get_user_model()` asks Django "whichever model is in charge of accounts, give me that one". If the project ever swapped in a custom user model, this line keeps working while `from django.contrib.auth.models import User` would quietly point at the wrong table.

THE SURPRISE

You log in with the **phone number** — not the username, not the email. That is why the phone lives on `Profile`, and why the login view searches there first.

The second table, in full

backend/backend/models.py

```
class Profile(models.Model):
    ADMIN    = "admin"
    TEACHER  = "teacher"
    STUDENT  = "student"

    ROLE_CHOICES = [
        (ADMIN,    "Admin"),
        (TEACHER,  "Teacher"),
        (STUDENT,  "Student"),
    ]

    user = models.OneToOneField(User, on_delete=models.CASCADE)
    phone = models.CharField(max_length=20, unique=True)
    role = models.CharField(max_length=10, choices=ROLE_CHOICES,
                           default=STUDENT)
```

`unique=True` does real work

The login view runs `Profile.objects.get(phone=phone)` — **get**, singular. If two accounts shared a number, that call raises `MultipleObjectsReturned` and **nobody could sign in**. The uniqueness rule is load-bearing, not tidiness.

`default=STUDENT` is a policy

An account with nothing specified is the **least powerful** kind. Defaults should fail towards less access, never more.

`on_delete=CASCADE`

Delete the User and the Profile goes with it. A phone number and role belonging to a deleted account are meaningless leftovers.

Your brand new superuser cannot log in

WHY

`createsuperuser` belongs to Django. It knows nothing about your Profile model, so it creates the **User row alone**. No phone number — and the login screen asks for a phone number.

Nothing is broken. The row it needs simply is not there yet. Fix it once, by hand, in the Django admin: Profiles → Add profile → pick your user, type a phone, choose role **Admin**.

```
python manage.py createsuperuser
python manage.py runserver 8001
# then http://127.0.0.1:8001/admin/
```

backend/backend/permissions.py — the code plans for this

```
def role_of(user):
    if not user or not user.is_authenticated:
        return None

    if user.is_superuser:
        return Profile.ADMIN

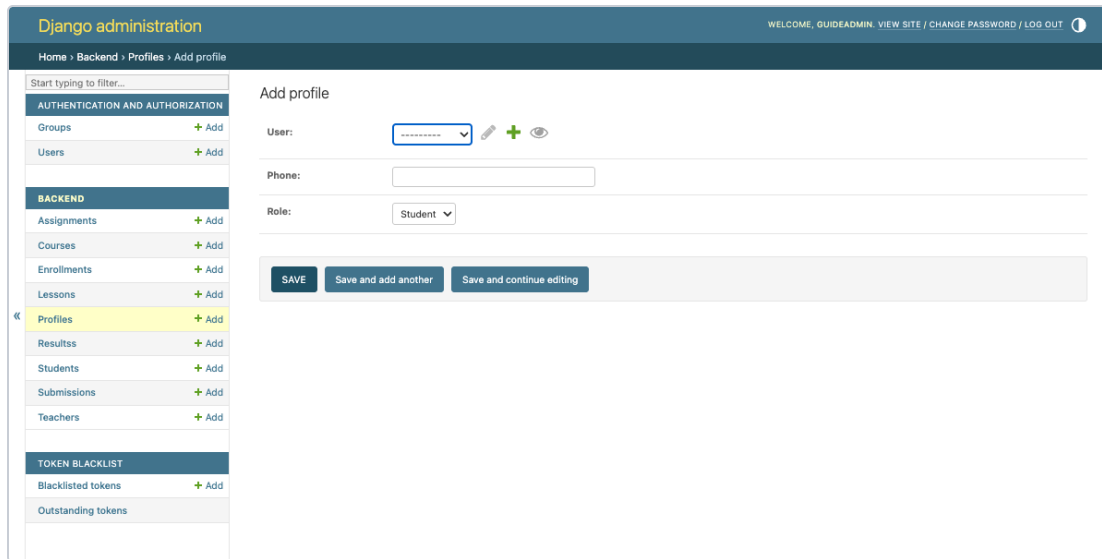
    profile = (Profile.objects.filter(user=user)
               .only("role").first())

    if profile is None:
        return Profile.STUDENT

    return profile.role
```

Read the last branch as a **policy**, not a fallback: when the code is unsure who you are, it gives you **less**, not more.

Three fields, and the account can finally log in



The screenshot shows the Django administration interface. The top navigation bar includes the Django logo, the title 'Django administration', and user information. The left sidebar contains a navigation menu with categories like 'AUTHENTICATION AND AUTHORIZATION', 'BACKEND', and 'TOKEN BLACKLIST'. The 'Profiles' link is highlighted. The main content area shows the 'Add profile' form with three fields: 'User' (a dropdown menu), 'Phone' (a text input), and 'Role' (a dropdown menu with 'Student' selected). Below the fields are three buttons: 'SAVE', 'Save and add another', and 'Save and continue editing'.

Django admin → Profiles → Add profile. The phone number typed here is what you type on the login screen.

Checkpoint

Start the React app, go to `/login`, sign in with that phone number and your superuser password. Land on the dashboard and **both halves of an account exist and are joined correctly**.

If you get "Invalid phone or password"

The phone in the Profile does not match what you typed. Check it character by character in the admin — a leading zero or a space is the usual culprit.

WHY THE ADMIN LETS YOU IN WITH NO PHONE

Django's `/admin/` uses the **cookie-and-session** mechanism from the last slide, not tokens. Different door, different lock.

PART TWO

Making an account

- Why there is **no public sign-up** here
- What a serializer is, and what `write_only` protects
- `create_user` vs `create` — the classic disaster
- What actually happens to a password
- Why one error message is deliberately vague



A student does not make their own account

An **admin** makes it for them and picks the role at the same time.
There is no sign-up link on the login screen.

frontend/src/App.jsx

```
{/* /register used to be a public page. There is no  
  public sign-up any more: an admin creates accounts  
  at /accounts, inside the app. */}  
<Route path="/register"  
  element={<Navigate to="/login" replace />} />
```

Why

Because of what a role is worth. In a school system, the difference between a student account and a teacher account is **the difference between reading grades and setting them**.

If anybody could sign up, then either everyone starts as a student and an admin promotes them one by one anyway — or the role is self-declared, which is no protection at all.

THE TRADE-OFF, AND IT IS REAL

Somebody has to be an admin before anybody can be anything. That is the chicken-and-egg problem we just solved with `createsuperuser` plus a hand-made Profile.

From a web address to the code that answers it

backend/lms/urls.py

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('backend.urls')),
]
```

backend/backend/urls.py

```
urlpatterns = [
    path('login/', LoginView.as_view()),
    path('register/', RegisterView.as_view()),
    path('profile/', ProtectedView.as_view()),
    path('change-password/', ChangePasswordView.as_view()),
    path('password-reset/', PasswordResetRequestView.as_view()),
    path('password-reset-confirm/',
        PasswordResetConfirmView.as_view()),
]
```

`api/` + `register/` = `/api/register/`. Those six addresses are the entire subject of today.

backend/backend/views.py

```
class RegisterView(generics.CreateAPIView):
    """Create an account. Admins only."""

    queryset = User.objects.all()
    serializer_class = RegisterSerializer
    permission_classes = [IsAdmin]
```

Three attributes and no method bodies. `CreateAPIView` is a ready-made DRF class that knows how to answer a POST by creating something. You tell it **what** to create with and **who** may do it; it supplies the rest.

A serializer is a translator that works both ways

```
JSON over the web      Rows in the database
{"phone": "0171...",    backend_profile
"role": "teacher"}      user_id | phone | role
                        ^
                        |
                        | COMING IN
+----- validate -----+
|   check every field is present,
|   the right type, acceptable
|
+----- GOING OUT -----+
|   choose which fields the
|   outside world may see
```

Coming in

Take the JSON the browser sent, check it, then create or update rows. **This is where validation lives.**

Going out

Take rows and turn them into JSON, choosing which fields are allowed to be seen.

That second half matters as much as the first. **The password column is never in the outgoing JSON** — and that is a decision written down in the serializer, not something that happens by luck.

The four fields written out by hand

backend/backend/serializers.py

```
class RegisterSerializer(serializers.ModelSerializer):
    phone = serializers.CharField(required=True,
                                  write_only=True)
    first_name = serializers.CharField(required=True)
    email = serializers.EmailField(required=True)
    role = serializers.ChoiceField(
        choices=Profile.ROLE_CHOICES,
        required=False,
        default=Profile.STUDENT,
    )

    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password',
                  'phone', 'first_name', 'last_name', 'role']
        read_only_fields = ['id']
        extra_kwargs = {'password': {'write_only': True}}
```

`ModelSerializer` means "look at the model in `Meta` and work out most of the fields yourself". Because `model = User`, five fields come free with the right types and lengths.

FIELD	WHY IT IS WRITTEN OUT
-------	-----------------------

phone	Not a User column at all — it belongs to <code>Profile</code> . Declaring it here lets it arrive in the same request.
-------	---

first_name	Django's version allows blank. This app insists on one.
------------	---

email	Needed later by the password reset flow, so it cannot be optional.
-------	--

role	Also a Profile column. <code>ChoiceField</code> restricts it to three known values — anything else is rejected before any row is written.
------	---

`write_only` — may come in, never goes out

Sent to `POST /api/register/`

```
{
  "username": "rina",
  "email": "rina@example.com",
  "password": "correct-horse-9",
  "phone": "01711110002",
  "role": "teacher",
  "first_name": "Rina",
  "last_name": "Haque"
}
```

Received — `201 CREATED`

```
{
  "id": 8,
  "username": "rina",
  "email": "rina@example.com",
  "first_name": "Rina",
  "last_name": "Haque",
  "role": "teacher"
}
```

LOOK AT WHAT IS MISSING

No `password`. No `phone`. Both were `write_only`. Without that keyword, **creating an account would answer with the password in the response.**

Turning crashes into sentences

backend/backend/serializers.py

```
def validate_phone(self, value):
    if Profile.objects.filter(phone=value).exists():
        raise serializers.ValidationError(
            "This phone number is already registered.")
    return value

def validate_username(self, value):
    if User.objects.filter(username=value).exists():
        raise serializers.ValidationError(
            "That username is already taken.")
    return value

def validate_password(self, value):
    validate_password(value)  # Django's own rules
    return value
```

Define a method called `validate_<fieldname>` and DRF calls it automatically with that field's value. Return the value to accept it; raise `ValidationError` to reject it with a message.

Why bother — the database already enforces this

Both columns are already unique, so the database would refuse anyway. Doing it here turns a `500` crash into a `400` with a sentence a human can act on.

The database protects the data. The serializer protects the person using the form.

```
{"phone":    ["This phone number is already registered."],
 "password": ["This password is too common."]}
```

Every message is a **list** — one field can fail more than one rule at once.

All of it, or none of it

backend/backend/serializers.py

```
@transaction.atomic
def create(self, validated_data):
    phone      = validated_data.pop('phone')
    email      = validated_data.pop('email')
    role       = validated_data.pop('role', Profile.STUDENT)
    first_name = validated_data.pop('first_name', '')
    last_name  = validated_data.pop('last_name', '')

    user = User.objects.create_user(
        username = validated_data['username'],
        email     = email,
        password  = validated_data['password'],
        first_name = first_name,
        last_name  = last_name,
    )
    Profile.objects.create(user=user, phone=phone, role=role)
    return user
```

Read the decorator first

`@transaction.atomic` wraps everything inside in a database transaction: **either all of it is saved or none of it is.**

WITHOUT IT

A failure halfway through leaves a User with **no Profile**: an account with a password and no way to log in — and a username now taken, so the admin cannot even retry with the same name.

Atomic makes the pair of rows an **all-or-nothing unit**, which is exactly what they are.

One missing suffix, one classic disaster

	<code>USER.OBJECTS.CREATE(...)</code>	<code>USER.OBJECTS.CREATE_USER(...)</code>
What it does with password	Stores the string exactly as given	Hashes it first , stores the hash
Result in the database	<code>opensesame</code>	<code>pbkdf2_sha256\$1000000\$...</code>
Can the person log in?	NO the check compares a hash to plain text	YES
Readable by anyone with DB access?	Yes. This is the classic disaster.	No

WHY IT IS SO HARD TO SPOT

It fails in a way that looks like a **login bug** rather than a security bug: the account is created, everything reports success, and then the password never works. If you ever see that symptom, look for `create` where `create_user` should be.

Two details worth copying into your own projects

The screenshot shows a web application interface for an LMS. On the left is a sidebar with a menu containing: Dashboard, Teachers, Students, Courses, Enrollments, Lessons, Assignments, Submissions, Results, and Accounts (which is highlighted). At the bottom of the sidebar, there is a user profile for 'nadia' with an 'Admin' role and a 'Log out' button. The main content area is titled 'Accounts' with a subtitle 'Create a login for a student, a teacher, or another admin.' Below this is a 'New account' form. The form has the following fields: 'First name' (Rina), 'Last name' (Chowdhury), 'Username' (rina), 'Email' (rina@lms.test), 'Phone' (01700000004), and 'Password' (masked with dots). There is a 'Role' dropdown menu currently set to 'Teacher — course material, enrollments and...'. Below the form, a green banner states: 'They sign in with the phone number, not the username.' At the bottom of the form are two buttons: 'Create account' (in blue) and 'Clear' (in grey).

The green banner repeats the phone number on purpose — that is what the new person signs in with, and it is the detail everyone forgets.

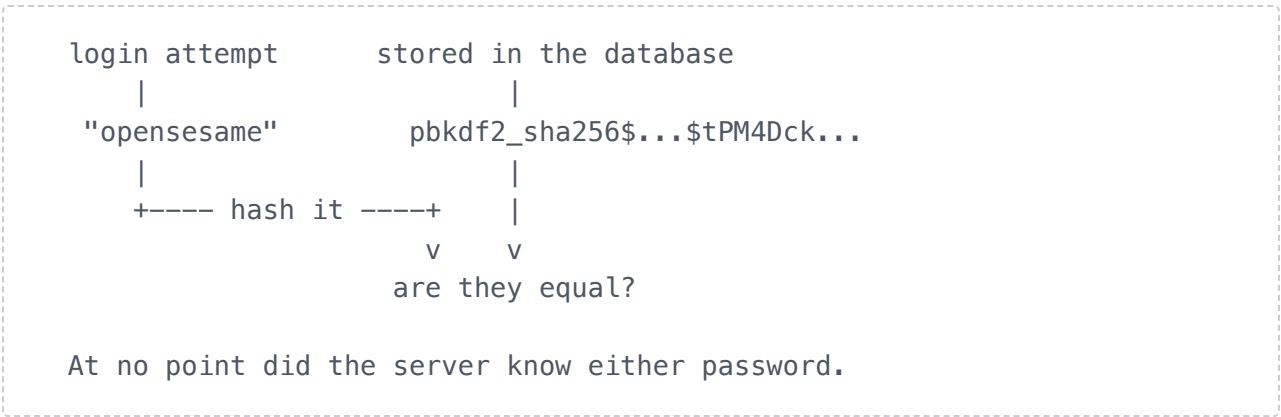
frontend/src/pages/Accounts.jsx

```
await register({ username, email, password, phone,
                  role, first_name, last_name });

// Deliberately not logged in as the new person: the
// admin who filled this in stays signed in as themselves.
setNotice(
  `Account created for ${username} as a ${role}. ` +
  `They log in with the phone number ${phone}.`
);
```

- The app does **not** log you in as the account you just created.
- The success message **repeats the phone number**, because the admin is about to have to tell somebody how to log in.

It is that **people reuse passwords**. The account in your teaching LMS very probably shares a password with somebody's email — and their email is the master key to everything else they own.



Three properties, plus one that surprises people

1 • Deterministic

The same input always gives the same output. That is what makes checking possible at all.

2 • One-way

Given the output there is **no calculation** that recovers the input. The only attack is guessing, hashing, comparing.

3 • Avalanche

A tiny change to the input changes the output completely. No partial credit, no "warm" hashes.

4 • For passwords only — it must be **slow on purpose**

A fast hash lets an attacker who steals the database try **billions of guesses a second**. A password hash is deliberately built to take a noticeable fraction of a second: nothing when you log in once, ruinous when you are running a dictionary.

Django 5.2 default: PBKDF2-SHA256 at **1,000,000 iterations**.

Four parts, cut by \$

```
pbkdf2_sha256$1000000$z5Chg0kZql1bmLAAHi3Ws7$tPM4DckZXkyDDVZmZHe4uV7lyF/5090x6GKw+i9ZlxY=
└─ algorithm ─┐ └─ iters ─┐ └─ salt ─┐ └─ hash ─┐
```

PART	MEANING
algorithm	Which hash function was used. Django 5.2's default.
iterations	Run a million times over. This is the deliberate slowness.
salt	Random text generated when the password was set.
hash	The actual result.

WHY STORE THE SETTINGS TOO

Each row **remembers how it was made**. That is what lets Django raise the iteration count in a future version without breaking anybody — when you next log in, Django notices your row is below the current standard and quietly re-hashes at the new setting.

The salt, and why it is not a secret

```
pbkdf2_sha256$1000000$z5Chg0kZq11bmLAAHi3Ws7$tPM4DckZXkyDDVZmZHe4uV7lyF/5090x6GKw+i9ZlxY=  
pbkdf2_sha256$1000000$gP905ZrDz5bM00XoM0RukX$pzjvPhl9jFYCKhBCjD6Ya5doVL9W1o2xj018WJ5LQpg=
```

Same password. Completely different stored strings, because each got a fresh random salt mixed in before hashing. Same input *and* same salt still gives the same output — which is why the salt has to be stored in plain sight. It is not a secret. It has a different job.

Job 1 · Identical passwords stop looking identical

Without salt, everyone who chose **123456** would have the same row, and one glance down the column tells an attacker which accounts to try first.

Job 2 · Precomputed tables become useless

Attackers keep enormous lists of common passwords already hashed. Those lists are worthless against a salt they have never seen — **every password must now be attacked separately.**

All of it is invisible from the code

```
user = User.objects.create_user(
    username="rina", password="correct-horse-9")

# on an account that already exists:
user.set_password("a-new-one")
user.save()      # set_password does NOT save.

user.check_password("correct-horse-9")  # True
user.check_password("Correct-horse-9")  # False
```

CLASSIC TRAP

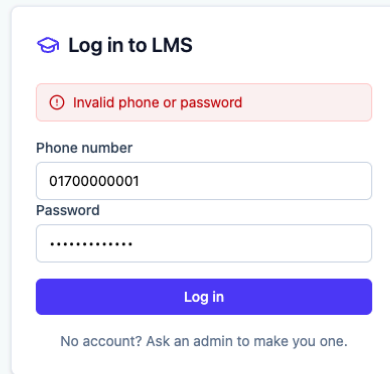
`set_password()` changes the object **in memory only**. Without `user.save()` on the next line, the new password is forgotten when the request ends and the old one still works — **with no error anywhere**.

backend/lms/settings.py — AUTH_PASSWORD_VALIDATORS

VALIDATOR	REFUSES
UserAttributeSimilarity	A password too close to the username, email or name
MinimumLength	Fewer than 8 characters
CommonPassword	Anything in Django's list of 20,000 most common
NumericPassword	Digits only, like 19283746

Called from all three places a password can be set: register, change, reset. **A rule enforced in two of three places is a rule that does not exist.**

One message for two different failures



The screenshot shows a login form titled "Log in to LMS" with a graduation cap icon. Below the title is a red error message box that says "Invalid phone or password". The form has two input fields: "Phone number" with the value "01700000001" and "Password" with masked characters ".....". A blue "Log in" button is at the bottom, and a link below it says "No account? Ask an admin to make you one."

```
try:
    profile = Profile.objects.get(phone=phone)
    user = profile.user
except Profile.DoesNotExist:
    return Response({'error': 'Invalid phone or password'},
                    status=400)

if not user.check_password(password):
    return Response({'error': 'Invalid phone or password'},
                    status=400)
```

The view really does have **two separate failure paths** — no Profile with that phone number, and a Profile whose password did not match — and it deliberately answers both with the same sentence and the same status code.

Account enumeration

If the messages differed — "no such phone number" vs "wrong password" — anyone could feed the endpoint a list of phone numbers and learn **which ones have accounts**, without guessing a single password.

That turns a login form into a **directory of your users**. One vague message closes it.

The same reasoning appears again in the forgotten-password flow, later today.

PART THREE

Login, and the token

- Thirty lines of Python, in order
- Why `AllowAny` on the login endpoint is compulsory
- Taking a JWT apart with nothing but a text editor
- Signed, **not** encrypted — and what that costs
- The eight-hour decision, written down honestly



The form has one job

Note the line underneath: there is no sign-up link, because of Part 2.

frontend/src/pages/Login.jsx

```
async function handleSubmit(event) {
  event.preventDefault();
  setError("");
  setIsSaving(true);

  try {
    // The backend looks you up by phone, not username.
    await login(phone, password);
    navigate(goBackTo, { replace: true });
  } catch (problem) {
    setError(problem.message);
  } finally {
    setIsSaving(false);
  }
}
```

`event.preventDefault()` stops the browser doing what HTML forms normally do — reload the whole page. This app handles the submission itself in JavaScript, so the page must stay put. **Every React form in the project starts with that line.**

Thirty lines, and you have seen most of them

```
class LoginView(APIView):
    permission_classes = [AllowAny]

    def post(self, request):
        serializer = LoginSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        phone = serializer.validated_data['phone']
        password = serializer.validated_data['password']

        try:
            profile = Profile.objects.get(phone=phone)
            user = profile.user
        except Profile.DoesNotExist:
            return Response({'error': 'Invalid phone or password'},
                            status=400)

        if not user.check_password(password):
            return Response({'error': 'Invalid phone or password'},
                            status=400)

        tokens = get_tokens_for_user(user)
        return Response({
            'message': 'Login successful',
            'user_id': user.id,
            'username': user.username,
            'role': role_of(user),
            'tokens': tokens,
        })
```

STEP	WHAT IT DOES
1 · shape	<code>is_valid(raise_exception=True)</code> — "check it, and if it is wrong, stop here and answer 400 with the reasons". No <code>if</code> needed; the exception ends the method.
2 · lookup	The search starts in <code>Profile</code> , not <code>User</code> . Then <code>profile.user</code> follows the one-to-one link.
3 · password	One line. Everything below it is the exact moment authentication succeeds.
4 · tokens	<code>RefreshToken.for_user(user)</code> signs two strings. Nothing is written to any table.

ONE HABIT

Read from `validated_data`, never `request.data` — that keeps unchecked input out of your logic.

Without **AllowAny**, you must log in before you can log in

backend/lms/settings.py

```
REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
    ),
}
```

Every endpoint in this project demands a signed-in caller by default.

Fail closed

Out of the box, DRF allows *anyone*. A view added in a hurry with no `permission_classes` would be open to the world and nothing would warn you. Flipping the default means **the forgetful case is the locked case**.

VIEW	OPENS UP WITH
LoginView	AllowAny
PasswordReset (both)	AllowAny
RegisterView	IsAdmin – <i>stricter</i>

This one line is the best security decision in the project.

The tokens are **nested**

```
{
  "message": "Login successful",
  "user_id": 7,
  "username": "nadia",
  "role": "teacher",
  "tokens": {
    "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6...\"",
    "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6..."
  }
}
```

WHERE EVERYONE LOSES TEN MINUTES

Most JWT tutorials answer with `access` at the **top level**, so most copied code reads `data.access` and gets `undefined`. Here it is `data.tokens.access`.

The role is included

Not because the frontend is trusted with it — so the sidebar can hide links a student cannot use. **The server checks the role again on every request regardless.**

Nothing was written to any table

At the end of a successful login this app **has not recorded that you logged in**. The proof went out the door with you.

The refresh token is sent, then dropped

This backend has no refresh endpoint. The frontend stores only the access token. We come back to what that costs.

Browser, Django, database



A signed note, not a claim ticket

The session approach gives you a numbered claim ticket and keeps a ledger at the desk.

The token approach gives you a note that reads "**the bearer of this note is employee 7, valid until 6pm**", signed by the manager — and keeps **no ledger at all**.

- **Anyone can read the note.**
- **Nobody can alter it**, because the signature would stop matching.
- Anyone *holding* it is treated as employee 7 — which is why the word in the HTTP header is literally **Bearer**.

Three parts, separated by full stops

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJ0b2t1b190eXB1IjoieYWNjZXNzIiwiZXhwIjoxNzg1
ODY4ODU0LCJpYXQiOjE3ODU4NDAwNTQsInVzZXJfawQi
Ojd9
.N8vyuuXgIBYFPG5cCbQz27dPnTs7Bpy5QG9WUW_VQrY
```

header

payload

signature

The first two are JSON that has been **base64url encoded** — writing text using only letters, digits, **-** and **_** so it survives being put in a header.

Almost every JWT starts with **eyJ** — those are the base64 letters for **{**.

What is actually inside

part one – the header

```
{"alg": "HS256", "typ": "JWT"}
```

How the token was signed. Nothing about you.

part two – the payload

```
{
  "token_type": "access",
  "exp": 1785868854,
  "iat": 1785840054,
  "jti": "d8d07c2f5b6b4fd99dfd56f58985aff7",
  "user_id": 7
}
```

CLAIM	WHAT IT DOES HERE
token_type	Stops a refresh token being used where an access token is expected
exp	When it stops working, in seconds since 1 Jan 1970. Exactly 28,800 after iat — eight hours
iat	When it was signed
jti	A unique id for this one token. What the blacklist tracks
user_id	The whole of what identifies you. This single number is how the server knows who you are

NOTE WHAT IS NOT IN THERE

No password, no phone number, and **no role**. The server looks the role up fresh on every request — which is exactly why demoting someone in the Django admin takes effect *immediately*, not in eight hours.

Not encrypted. Signed.

A JWT is readable by anybody who has it

Encoding is not encryption. If you put a person's email, salary or role in a token, **everyone holding that token can read it** — including the person themselves, and anyone who steals it.

What the signature guarantees is integrity

Not secrecy. Nobody can change the contents. Edit `"user_id": 7` to `"user_id": 1`, re-encode, and the signature no longer matches. The server rejects it, because **only someone holding the secret key can produce a valid signature**.

So the rule for a payload is simple: **facts you would not mind the holder reading, and nothing you would mind them knowing**.

backend/lms/settings.py

```
SECRET_KEY = 'django-insecure-@hg+%+(*w2m0jh=8$ksn...'
```

Everything rests on that string. Anybody who knows it can sign a token claiming to be `user_id: 1`, and your server will believe it completely — **because believing signatures is the entire mechanism**.

DO NOT SHIP THIS

A `SECRET_KEY` committed to Git is a **real, exploitable vulnerability**, not a style issue. On a real deployment it must be read from an environment variable and never appear in the repository.

Changing it invalidates every token at once — worth knowing both as a consequence and as a blunt emergency tool.

A trade, written down honestly

	ACCESS TOKEN	REFRESH TOKEN
Sent with	Every request, in the header	Only to the refresh endpoint
Lifetime here	8 hours	7 days
Normal lifetime	5 – 15 minutes	Days or weeks
If stolen	Damage limited by the clock	Bad — it mints fresh access tokens
Used in this app	YES constantly	NO received and discarded

```
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(hours=8),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "AUTH_HEADER_TYPES": ("Bearer",),
}
```

The library default is 5 minutes

Without a refresh endpoint, five minutes throws you back to the login screen **in the middle of typing a lesson description**. So the author widened the access token to eight hours instead of building the refresh flow.

What it costs: a stolen token is useful for up to eight hours instead of five minutes.

What it buys: an app usable for a day's work, and one less endpoint to build.

THE HABIT WORTH TAKING

When you loosen a security setting, **write down what you traded and why**, the way that settings file does.

`AUTH_HEADER_TYPES` is the third line and the one you meet in practice: write `Token <t>` or `JWT <t>` and this server treats you as a stranger.

You cannot un-sign a signed note

The server keeps **no list of valid tokens**, so there is nothing to delete. Once an access token is out there, it works until `exp` passes, and nothing short of changing `SECRET_KEY` can stop it.

Compare with sessions, where logging somebody out is one `DELETE`.

Three places this project runs into it — each handled honestly

Logging out

The frontend deletes its copy. **The token itself stays valid until it expires.** There is no logout endpoint, because there is nothing useful for one to do.

Changing your password

Refresh tokens are blacklisted; the access token **cannot be**. So the app signs you out on the spot and makes you log back in.

Demoting someone

This one is fine, **by design**. The role is not in the token, so the change applies to the very next request.

PART FOUR

Staying in, and what it gets you

- Where the token lives between requests
- One wrapper function that guarantees the header
- `ProtectedRoute` is a courtesy, **not** a security control
- How a header becomes `request.user`
- `401` vs `403` — the fastest debugging skill in this subject



The server forgot the whole thing. Now it is the browser's job.

FILE	RESPONSIBILITY
src/api.js	Every call to Django. Stores the token, attaches it, turns error responses into readable messages.
src/AuthContext.jsx	Holds "am I logged in, and who am I" so any component can ask.
src/components/ProtectedRoute.jsx	Stands in front of pages that need a signed-in visitor.

Plus a two-line helper, `src/auth.js`, holding the context object and the `useAuth` hook — separate so `AuthContext.jsx` exports only a component, because Vite's fast refresh gives up on any file that mixes the two.

frontend/src/api.js

```
const TOKEN_KEY = "lms_access_token";
const USER_KEY = "lms_user";

export function getToken() {
  return localStorage.getItem(TOKEN_KEY);
}

export function saveSession(token, user) {
  localStorage.setItem(TOKEN_KEY, token);
  localStorage.setItem(USER_KEY, JSON.stringify(user));
}

export function clearSession() {
  localStorage.removeItem(TOKEN_KEY);
  localStorage.removeItem(USER_KEY);
}
```

The token has to survive you pressing F5. A JavaScript variable does not. `localStorage` is a small store of text the browser keeps **per website, permanently**, until code or the user clears it.

How does the server know it is me?

```
async function request(path, options = {}) {
  const { auth = true, ...rest } = options;

  const headers = { ...(rest.headers ?? {}) };
  if (rest.body !== undefined) {
    headers["Content-Type"] = "application/json";
  }

  const token = getToken();
  if (auth && token) {
    headers.Authorization = `Bearer ${token}`;
  }

  let response;
  try {
    response = await fetch(BASE_URL + path, { ...rest, headers });
  } catch {
    throw new Error("Could not reach the server...");
  }
  ...
}
```

Every call in the app goes through this one function — which is exactly why it can guarantee the header is there.

- `auth = true` is the **default**. Only three calls opt out: login, and the two halves of password reset. Defaulting to secure and opting out deliberately is the same instinct as `DEFAULT_PERMISSION_CLASSES` on the server.
- The header is spelled `Authorization: Bearer <token>`. Note **the space and the capital B**.
- The `catch` fires when the request **never reached a server at all** — Django down, wrong port, no network. That is different from a request that arrives and is refused.

Both ends must name the same port

```
// frontend/src/api.js
const BASE_URL = import.meta.env.VITE_API_URL
  ?? "http://127.0.0.1:8001/api";
```

```
# README
python manage.py runserver 8001
```

Django's own default is **8000**. Start it there by habit and the frontend calls **8001**, nothing answers, and you get the frontend's own "could not reach the server" message rather than a status code.

Fix it in one of two places

```
python manage.py runserver 8001
```

— or, to use any other port —

```
# frontend/.env
VITE_API_URL=http://127.0.0.1:8000/api
```

AND THEN

Restart the Vite dev server. Environment variables are read at startup — editing `.env` with Vite already running changes nothing.

Surviving a page refresh

frontend/src/AuthContext.jsx

```
export function AuthProvider({ children }) {  
  // The token is read back out of localStorage on the  
  // first render so a page refresh does not log you out.  
  const [user, setUser] =  
    useState(() => getSavedUser());  
  const [isLoggedIn, setIsLoggedIn] =  
    useState(() => Boolean(getToken()));
```

```
  async function logIn(phone, password) {  
    const data = await loginRequest(phone, password);  
  
    // The tokens are nested. data.access does not exist.  
    const nextUser = { id: data.user_id,  
                       username: data.username,  
                       role: data.role };  
    saveSession(data.tokens.access, nextUser);  
  
    setUser(nextUser);  
    setIsLoggedIn(true);  
  }  
}
```

Both `useState` calls take a **function**, not a value. React calls it once, on the very first render — so the app boots by asking localStorage "was somebody signed in here?" and starts in the right state **with no flicker** through a logged-out screen.

Why both localStorage *and* React state

localStorage survives a reload but React does not watch it. React state redraws the screen but vanishes on reload.

Write one without the other and you get the two classic bugs: **an app that still looks signed out until you refresh, or one that looks signed in until you refresh.**

ProtectedRoute is a convenience, not a security control

```
// `role`, when given, is the one role allowed through.  
// Hiding a link in the sidebar is not enough on its own  
// – somebody can always type the URL.  
export default function ProtectedRoute({ role, children }) {  
  const { isLoggedIn, user } = useAuth();  
  const location = useLocation();  
  
  if (!isLoggedIn) {  
    return <Navigate to="/login" replace  
      state={{ from: location.pathname }} />;  
  }  
  
  // The role arrives a moment after the token when a saved  
  // session is being topped up from /profile/.  
  if (role && !user?.role) return null;  
  
  if (role && user.role !== role)  
    return <Navigate to="/" replace />;  
  
  return children;  
}
```

Three checks, in order. Not signed in → `/login`, with a note of where **you were going**, which is the `goBackTo` the login form used. Role required but not known yet → draw nothing for a moment rather than guess wrong. Wrong role → back to the dashboard.

Read this carefully

It runs **in the browser**, on code the visitor could edit. It exists so people are not shown doors that will not open for them.

The thing that actually stops a student creating accounts is

`isAdmin` **on the server**.

How a header becomes a person

backend/lms/settings.py

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication",
        ".JWTAuthentication",
    ),
}
```

You never wrote code to read the `Authorization` header. **One setting arranged it.**

THE PRICE OF NOT PUTTING THE ROLE IN THE TOKEN

Step 5 is a database read on **every request**. In exchange, a change made in the Django admin takes effect on the very next call rather than in eight hours.

Before any view runs, in order

- Look for an `Authorization` header. Nothing there? Stop — the caller is anonymous.
- Check it starts with `Bearer`. Take the rest as the token.
- **Verify the signature** with `SECRET_KEY`. Altered in any way? Reject.
- Check `exp` against the clock. Expired? Reject.
- Read `user_id` out of the payload, load that row from `auth_user`, and put it on the request.

After which every view can simply write `request.user`. When nobody is signed in it is Django's `AnonymousUser`, whose `is_authenticated` is `False` — which is why the permission code checks that flag rather than testing for `None`.

Four classes, one shared base

```
class RoleWritePermission(BasePermission):
    """Signed-in accounts read. Only these roles write."""
    roles_that_may_write = ()
    message = "Your role does not allow this change."

    def has_permission(self, request, view):
        if not request.user or \
            not request.user.is_authenticated:
            return False

        if request.method in SAFE_METHODS:
            return True

        return role_of(request.user) in \
            self.roles_that_may_write
```

A permission class is an object with a `has_permission()` method returning `True` or `False`. DRF calls it before the view runs. **That is the entire interface.**

`SAFE_METHODS` is DRF's tuple `( "GET", "HEAD", "OPTIONS" )` — the methods that only look. Anything else changes something and falls through to the role check.

CLASS	MAY WRITE
IsAdmin	Admins only — and it blocks reading too
AdminWrites	admin
TeachingStaffWrites	admin, teacher
SubmissionWrites	admin, teacher — plus student POST only

Each carries its own `message`, which is why refusals in this app read like sentences rather than "Permission denied".

A student may hand work in — and nothing else

```
class SubmissionWrites(RoleWritePermission):
    """Same as above, except a student may hand work in.

    A student cannot edit or delete a submission, because
    nothing links a Student row to a login account — so the
    server has no way to tell whose work it is.
    """
    roles_that_may_write = (Profile.ADMIN, Profile.TEACHER)

    def has_permission(self, request, view):
        if (request.method == "POST"
            and request.user
            and request.user.is_authenticated
            and role_of(request.user) == Profile.STUDENT):
            return True
        return super().has_permission(request, view)
```

auth_user + backend_profile	backend_student
-----	-----
who can log in	who is in the school
username, password,	name, email,
phone, role	roll_number
(no link between them)	

A logged-in student is a **User**. A submission belongs to a **Student**. The server **cannot tell** whether the person POSTing a submission is the student named in it — so it allows the create and forbids everything else.

The general shape of a whole class of security bugs

The permission check can only be as precise as the data model allows. No cleverness in `permissions.py` can answer "is this your submission?" while the answer is not recorded anywhere.

Learn the meanings, ignore the labels

	401 UNAUTHORIZED	403 FORBIDDEN
Means	I do not know who you are	I know who you are, and no
Cause	No token, malformed token, bad signature, expired	Valid token, wrong role for this action
Body	<code>{"detail": "Authentication credentials were not provided."}</code>	<code>{"detail": "Only an admin can create accounts."}</code>
Fix	Log in again, or attach the header properly	Nothing you can do. Ask an admin
The app's response	Clears the session, back to <code>/login</code>	Shows the message in a red banner, stays put

THE NAMES ARE HISTORICAL AND UNHELPFUL

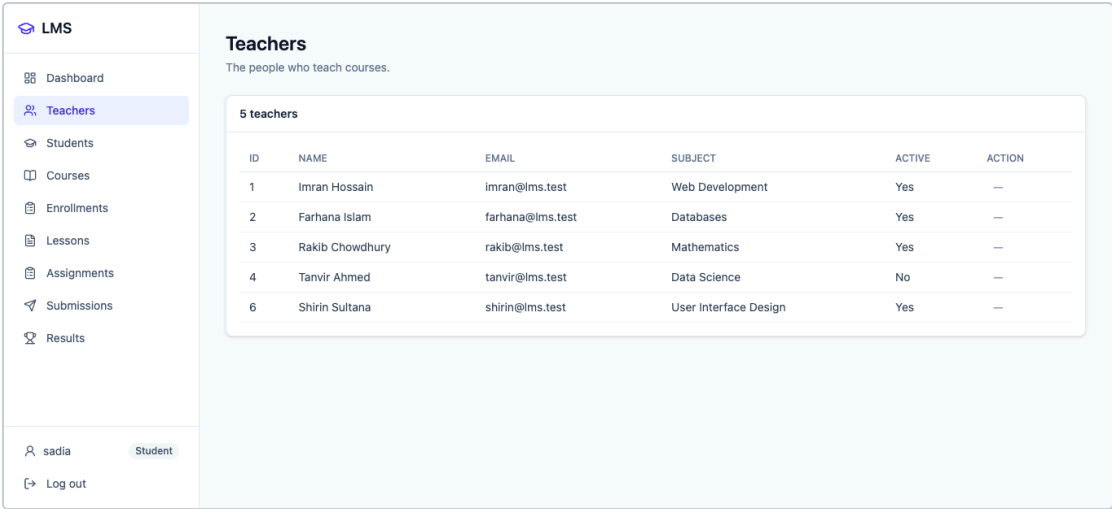
401 *"Unauthorized"* is the **authentication** failure. 403 *"Forbidden"* is the **authorisation** failure. That is the opposite of what the words suggest.

Who may do what

ENDPOINT	CLASS	ADMIN	TEACHER	STUDENT
/api/register/	IsAdmin	ALL	NOTHING	NOTHING
/api/login/	AllowAnonymous	Open to everyone, signed in or not		
/api/profile/	IsAuthenticated	Your own account only, whatever your role		
/api/change-password/	IsAuthenticated	Your own password only		
/api/password-reset/	AllowAnonymous	Open — you are locked out when you need it		
/api/teacher/ · /api/student/	AdminWrites	read + write	read	read
/api/course/ · lesson · assignment · enrollment · results	TeachingStaffWrites	read + write	read + write	read
/api/submission/	SubmissionWrites	read + write	read + write	read + create only

Two things stand out. Reading is open to every signed-in account everywhere — a student can list teachers and courses. And a student's *only* write in the entire API is handing in a submission.

One copy is a control. The other is a hint.



The Teachers page as a student. The list loads, because reading is open. No Add button, and dashes where edit and delete would be.

frontend/src/permissions.js

```
// A copy of the rules in backend/permissions.py, used only
// to decide which buttons are worth showing. The server is
// what actually enforces them, so a mistake here is a
// cosmetic bug, not a hole. Keep the two in step.
```

Duplicating rules is normally a mistake, so be precise about why it is right here — **the two copies do different jobs.**

	SERVER COPY	BROWSER COPY
Is a	control — decides what happens	hint — decides what is worth drawing
Get it wrong	a security hole	somebody sees a button that gives them a red banner

Go around the entire frontend, straight at the API

```
# 1 · knock without a token
curl -i http://127.0.0.1:8001/api/teacher/

HTTP/1.1 401 Unauthorized
{"detail":"Authentication credentials were not provided."}
```

```
# 2 · log in, keep the token
TOKEN=$(curl -s -X POST \
  http://127.0.0.1:8001/api/login/ \
  -H "Content-Type: application/json" \
  -d '{"phone":"01711110001","password":"admin-pass-1"}' \
  | jq -r .tokens.access)
```

```
# 3 · same request, one header added
curl -i http://127.0.0.1:8001/api/teacher/ \
  -H "Authorization: Bearer $TOKEN"

HTTP/1.1 200 OK
```

```
# 6 · as a student, try to create an account
curl -i -X POST http://127.0.0.1:8001/api/register/ \
  -H "Authorization: Bearer $STUDENT_TOKEN" ...

HTTP/1.1 403 Forbidden
{"detail":"Only an admin can create accounts."}
```

```
# 7 · break the token on purpose
curl -i http://127.0.0.1:8001/api/teacher/ \
  -H "Authorization: Bearer ${TOKEN}x"

401 - "Given token not valid for any token type"
```

The moment it clicks

The React app never draws that button for a student and the sidebar has no Accounts link — **but you just went around the entire frontend and the server refused anyway.** That is the difference between hiding a door and locking it.

PART FIVE

Passwords again, and the honest list

- Changing it vs resetting it — two different problems
- A reset token that is single-use **without storing anything**
- What this project deliberately does not do
- The eight sentences worth remembering



Change, and reset

	CHANGE	RESET
The situation	You know your password and want a new one	You have forgotten it and cannot get in
Are you signed in?	Yes	No — that is the whole problem
How you prove it is you	Type the current password	Read an email only you can read
Endpoint	/api/change-password/	/api/password-reset/ then -confirm/
Permission	IsAuthenticated	AllowAnonymous

THE SECOND ROW IS THE INTERESTING ONE

A reset endpoint **must** be open to anonymous callers, because a locked-out person has no token. That makes it the softest surface on the whole API — and everything odd about its design is a response to that fact.

A live session is not enough

```
def validate_current_password(self, value):
    # Knowing the old password is what makes this yours to
    # change. Without it, a borrowed browser tab would be
    # enough to take the account.
    if not self.context["request"].user.check_password(value):
        raise serializers.ValidationError(
            "That is not your current password.")
    return value

def validate(self, attrs):
    if attrs["new_password"] != attrs["confirm_password"]:
        raise ...("The two new passwords do not match.")
    if attrs["new_password"] == attrs["current_password"]:
        raise ...("The new password has to be different.")
    validate_password(attrs["new_password"],
                      user=self.context["request"].user)
    return attrs
```

Two kinds of validation method. `validate_<field>` checks one field on its own. Plain `validate` runs afterwards and can see *all* the fields at once — the only way to compare two of them.

Why ask for the current password at all?

If a live session were enough, walking away from an unlocked laptop would mean **losing the account permanently** rather than temporarily — the person at your desk could lock you out of your own login.

Asking for the current password means the attacker needs the secret, not just the chair.

The view operates on `request.user` and nothing else, so **there is no field anywhere that could aim it at another account**. And `set_password` is followed immediately by `save()`.

uid answers "who". token proves it.

```
uid = urlsafe_base64_encode(force_bytes(user.pk))
token = default_token_generator.make_token(user)

reset_link = f"{FRONTEND_PASSWORD_RESET_URL}" \
             f"?uid={uid}&token={token}"
```

uid is the primary key, base64url-encoded so it survives a URL. User 7 becomes **Nw**. **Nothing secret about it** — encoding, again, is not encryption.

token is not a JWT. Django builds a hash out of four ingredients, signed with **SECRET_KEY**:

- the user's primary key
- **the current password hash**
- the account's **last_login** timestamp
- the time the token was made

PROPERTY

WHY IT HOLDS

Single use

The password hash is an ingredient. Once the password changes, **the hash changes and the token no longer verifies**.

Expires

The creation time is an ingredient, checked against **PASSWORD_RESET_TIMEOUT** — **1 hour** here, vs Django's default of 3 days.

Unforgeable

Signed with **SECRET_KEY**, which the requester does not have.

The elegant bit

Nobody wrote code to mark the token as used. **It invalidates itself as a side effect of the thing it was for.**

AND THE SAME TRICK AS LOGIN

The endpoint answers "If an account exists with this email, a reset link has been sent." — **whether or not it does**. Anti-enumeration, one more time.

A teaching app, not a bank

Three settings that must move to the environment

`SECRET_KEY` in the file is **complete authentication bypass** for anyone who reads the repo. `DEBUG = True` returns a full traceback — file paths, local variables, settings — to any caller. And a real email address sits as a default next to `EMAIL_HOST_PASSWORD`.

No rate limit on login

Nothing counts failed attempts, nothing locks an account, no CAPTCHA. **This is the most likely way this app would actually be broken into** — and the cheapest to fix:

```
"DEFAULT_THROTTLE_CLASSES":
    ("rest_framework.throttling.AnonRateThrottle",),
"DEFAULT_THROTTLE_RATES": {"anon": "20/hour"},
```

The token in localStorage

Any JavaScript on the page can read it — including anything injected through an XSS hole. The alternative, an `HttpOnly` cookie, closes this and opens CSRF instead. **Neither choice is free; the difference is which attack you are prepared for.**

Also missing

Email verification. Two-factor. An audit trail of who changed a role. Account lockout or notification. A logout endpoint.

READ THAT LIST AS A CURRICULUM, NOT A COMPLAINT

Each item is a real system's real feature — and now you know exactly which problem each one exists to solve.

The clearest example in this codebase

```
no refresh endpoint
|
v
a 5-minute access token would be unusable
|
v
access token widened to 8 hours
|
v
a stolen token is useful for 8 hours
|
v
logout cannot cancel it, so logout is
  "delete our copy"
|
v
changing a password cannot cancel it either,
so the app signs you out and asks you to
  log in again
```

The fix is about ten lines

Adding `TokenRefreshView` from Simple JWT would let the access token drop to five minutes.

It *also* needs the frontend to catch a 401, call refresh, and retry the original request — **which is the real work, and the reason it was skipped.**

THE HABIT TO TAKE AWAY

Every security setting you loosen sets up the next compromise. Trace the chain **before** you loosen it, and write down what you traded.

The eight sentences worth keeping

- HTTP **forgets you** between requests, so proof travels with every one.
- You log in with the **phone number**, because that is what the Profile is found by.
- Passwords are **hashed, never stored**; the salt makes identical passwords look different.
- A JWT is **signed, not encrypted** — anyone holding it can read it.
- The **role is not in the token**, so a change in the admin applies to the next request.
- **401** means "I do not know you". **403** means "I know you, and no".
- The frontend's permission rules **hide buttons**; the server's rules are what **stop** anything.
- A signed note **cannot be taken back**, which is why logging out only deletes your copy.

Type it, do not skim it

Do these five, in order

- Hash one word twice in `manage.py shell` with `make_password`. Confirm the two strings differ and **both verify**. Explain to somebody else why that is not a contradiction.
- Log in with dev tools open. Find the `login/` request, read Payload and Response. **Decode the middle section of the token.**
- Copy `lms_access_token` out of localStorage, log out, paste it back, reload. You are signed in again **without a password**. Explain why that is not a bug.
- Run steps 1, 3, 6 and 7 of the curl walkthrough. Get yourself a real 401 and a real 403.
- Do the password reset end to end — the email prints into the `runserver` terminal. Then **try the same link twice**.

The five-minute checklist, when something breaks

- Is Django running, on the port the frontend calls?
- What is the **status code**? Network tab, or `-i` in curl.
- **Does the same call work in curl?** Yes → the bug is in the frontend. No → it is in the backend.
- Is there a token in localStorage, and is it whole? Check `exp`.
- Does the account have a Profile, with the role you expect?
- Anything in the `runserver` terminal?

Step 3 is the one to internalise. "Works in curl, fails in the browser" means a header, CORS or a URL problem. "Fails in both" means the answer is in `permissions.py` or a serializer.

END OF CLASS

Questions.

Full write-up: [docs/guide/lms-auth-guide.pdf](#) — twelve chapters, a glossary, and the whole flow on one page.

[Ch 10 – do it yourself in curl](#)

[Ch 11 – when it goes wrong](#)

[Ch 12 – what this project does not do](#)

[Appendix B – the whole flow on one page](#)